

Question & self study

Question: What is the shortcut to comment and uncomment a selected block of code in Visual Studio?

Answer:

In Visual Studio, you can use the following keyboard shortcuts to comment or uncomment a selected block of code:

- **Comment:** Ctrl + K, Ctrl + C
- **Uncomment:** Ctrl + K, Ctrl + U

Question: Explain the difference between a runtime error and a logical error with examples.

Answer:

A **runtime error** occurs while the program is running and causes the program to stop or throw an exception.

Example:

```
int x = 10;  
int y = 0;  
  
int result = x / y; // Runtime error: Division by zero
```

A **logical error** occurs when the program runs successfully, but produces an incorrect result because the logic is wrong.

Example:

```
int x = 10;  
int y = 20;  
  
int sum = x - y; // Logical error: should be x + y  
  
Console.WriteLine(sum);
```

In short:

- **Runtime Error:** The program fails during execution.
- **Logical Error:** The program runs, but gives the wrong output.

Question: Why is it important to follow naming conventions such as PascalCase in C#?

Answer:

Following naming conventions such as **PascalCase** in C# is important because it makes the code **clear, readable, consistent, and easier to maintain**. It also helps developers understand the purpose of classes, methods, and other identifiers quickly.

Question: Explain the difference between value types and reference types in terms of memory allocation.

Answer:

The main difference is how they are stored in memory:

- **Value Types:** Store the actual value directly. When assigned to another variable, a copy of the value is created. Examples: int, double, bool, char, and struct.
- **Reference Types:** Store a reference to an object in memory. When assigned to another variable, both variables can refer to the same object. Examples: class, array, and object.

Question: What will be the output of the following code? Explain why: int a = 2, b = 7; Console.WriteLine(a % b);

Answer:

The output will be:

2

Explanation:

The % operator returns the **remainder** after division.

Here:

```
int a = 2;
```

```
int b = 7;
```

```
Console.WriteLine(a % b);
```

$2 \div 7 = 0$ with a remainder of **2**, because 7 cannot fit into 2 even once.

So:

$2 \% 7 = 2$

Note: If the first number is smaller than the second number, the remainder is the first number.

Question: How does the && (logical AND) operator differ from the & (bitwise AND) operator?

Answer:

The main difference is what they operate on:

- **&& (Logical AND):** Used with Boolean conditions. It returns true only when **both conditions are true**. It also uses **short-circuit evaluation**, meaning the second condition may not be evaluated if the first is false.
- **& (Bitwise AND):** Used to perform an AND operation on the **individual bits** of integer values. When used with Boolean values, it evaluates both operands without short-circuiting.

Question: Why is explicit casting required when converting a double to an int?

Answer:

Explicit casting is required when converting a double to an int because the double may contain a **decimal part**, which will be lost when converted to an int.

For example:

```
double number = 10.5;
```

```
int result = (int)number;
```

```
Console.WriteLine(result);
```

Output:

10

The (int) tells C# that we **intentionally want to remove the decimal part**.

In short:

double → int requires explicit casting because **data may be lost**.

Question: What exception might occur if the input is invalid and how can you handle it

Answer:

If the user enters an invalid value, such as letters instead of a number, int.Parse() will throw a **FormatException**.

It can be handled using a try-catch block:

```
try
{
    int age = int.Parse(Console.ReadLine());
    Console.WriteLine("Age: " + age);
}
catch (FormatException)
{
    Console.WriteLine("Invalid input. Please enter a valid number.");
}
```

In short:

FormatException → occurs when the input cannot be converted to the required type.

Question: Given the code below, what is the value of x after execution? Explain why

```
int x = 5; int y = ++x + x++;
```

Answer:

The value of x after execution is:

7

Explanation:

```
int x = 5;
```

```
int y = ++x + x++;
```

The expression is evaluated from **left to right**:

1. `++x` → **Pre-increment**: x becomes 6, and the value used is 6.
2. `x++` → **Post-increment**: the value used is 6, then x becomes 7.
3. Therefore:

$$y = 6 + 6 = 12$$
$$x = 7$$

So the final values are:

$$x = 7;$$
$$y = 12;$$

2- what's the difference between compiled and interpreted languages and in this way what about Csharp?

Answer:

The main difference is how the code is translated and executed:

- **Compiled Languages:** The source code is translated into machine code or an intermediate code **before execution**. This usually makes execution faster.
- **Interpreted Languages:** The code is translated and executed **at runtime**, generally without producing a traditional standalone machine-code executable first.

What about C#?

C# is primarily a compiled language. C# source code is compiled by the C# compiler into **Intermediate Language (IL)**. The **.NET Runtime (CLR)** then uses **JIT (Just-In-Time) compilation** to convert the IL into native machine code during execution.

C# Source Code

↓

C# Compiler

↓

Intermediate Language (IL)

↓

CLR + JIT

↓

Native Machine Code

↓

Execution

3- Compare between implicit, explicit, Convert and parse casting

Method	Meaning	Example	Notes
Implicit Casting	Conversion happens automatically when it is safe.	<code>double x = 10;</code>	No casting syntax is required.
Explicit Casting	You manually specify the target type.	<code>int x = (int)10.5;</code>	May cause data loss.
Convert	Uses the Convert class to convert between compatible types.	<code>int x = Convert.ToInt32("10");</code>	Handles different types and provides several conversion methods.
Parse	Converts a string containing a valid value into a specific type.	<code>int x = int.Parse("10");</code>	Mainly used when you know the string contains a valid value; otherwise it can throw an exception.

5- what meant by Csharp is managed code

Answer:

When we say **C# is managed code**, it means that C# code runs under the control of the **.NET Runtime (CLR)** instead of directly managing the computer's resources.

The **CLR (Common Language Runtime)** manages important tasks such as:

- **Memory management and Garbage Collection (GC).**

- **Exception handling.**
- **Type safety.**
- **Security and runtime execution.**

For example:

```
int x = 10;
```

When objects are created, the **CLR manages their memory**, and the **Garbage Collector** automatically removes objects that are no longer needed.

In short:

Managed Code → **Code whose execution and memory management are controlled by the .NET Runtime (CLR).**

6- what meant by struct is considered like class before

Answer:

If the question means "**Was struct considered like class before?**", the important point is that in C#, both struct and class can be used to create custom types, but they behave differently.

- **struct** is a **Value Type**.
- **class** is a **Reference Type**.

For example:

```
struct Point
{
    public int X;
}
```

```
class Person
```



```
{  
    public int Age;  
}
```

When you assign a struct to another variable, its **value is copied**.

When you assign a class object to another variable, the **reference is copied**, so both variables can point to the same object.

In short:

struct → Value Type → copies the value.

class → Reference Type → copies the reference.

Self study :

1-Naming Conventions :

Naming Conventions

Choosing clear and expressive names for variables makes the code easier to understand and maintain. Naming conventions also help keep the code organized and consistent.

- **Expressive Names:** Names should clearly describe the purpose of the variable, such as `FirstName`.
- **camelCase:** The first word starts with a lowercase letter, and each following word starts with an uppercase letter. Commonly used in **JavaScript**.
 - Example: `firstName`
- **kebab-case:** Words are separated using hyphens, such as `first-name`. It is commonly used in **Angular**.
- **PascalCase:** Each word starts with an uppercase letter. This convention is commonly used in **C#**.
 - Example: `FirstName`

- **Underscore Naming:** An underscore can be used before a name, such as `_firstName`. This was mentioned as a naming convention to review further.

2-Manual Memory Management in C :

In **C**, memory management is handled manually by the programmer. The programmer is responsible for allocating memory when it is needed and releasing it when it is no longer required. Properly managing memory is important to prevent **memory leaks**, which happen when allocated memory is not released after it is no longer being used. Therefore, the programmer must carefully manage memory to ensure efficient use of system resources.

3- Stack Memory

The **Stack** is a region of memory used during program execution to store data related to method calls and local variables. In C#, value types are generally associated with stack allocation when they are local variables, and the stack has a predictable and structured nature.

The stack follows a **LIFO (Last In, First Out)** structure, meaning that the last item added to the stack is the first item removed. When a method is called, the required information is placed on the stack, and when the method finishes, that information can be removed automatically.

Because the stack is managed in a structured way, allocation and deallocation are generally fast and automatic. However, the exact storage location of a value in .NET depends on its context, so it is better not to assume that every value type always lives on the stack.

4- Heap Memory

The **Heap** is a memory area mainly used for storing objects created from **reference types**, such as objects created from classes. Unlike the stack, the heap is designed to handle objects that may have different sizes and lifetimes.

For example, when we create an object using the `new` keyword, memory is allocated for that object:

```
Person person = new Person();
```

The variable `person` holds a reference to the object, while the actual object is stored in the managed heap.

The heap is managed by the **CLR (Common Language Runtime)**. The CLR is responsible for allocating memory for objects and, through the **Garbage Collector (GC)**, automatically identifying objects that are no longer reachable and reclaiming their memory.

The notes describe the heap as having an unknown or variable size because different objects can contain different amounts of data.

5-Bitwise Operators

Bitwise operators are operators that work directly with the individual bits of a value. They are considered low-level operations because they allow the programmer to manipulate data at the binary level.

For example, the bitwise AND operator `&` compares the corresponding bits of two values. Other bitwise operations include:

- **`&` → Bitwise AND**
- **`|` → Bitwise OR**
- **`^` → Bitwise XOR**
- **`<<` → Left Shift**
- **`>>` → Right Shift**

Bitwise operations can be useful when working with flags, where individual bits can represent different settings or states. They are also used in situations that require direct manipulation of binary data or low-level operations.

6-Unchecked

The **unchecked** context in C# is related to how the program handles **numeric overflow**. An overflow happens when the result of an arithmetic operation is outside the range that the data type can store.

For example, an int can store values only within its specific range. If an operation produces a value outside that range, an overflow can occur.

By default, some operations may be performed without throwing an exception, depending on the context. The unchecked keyword explicitly tells C# that overflow checking should not be performed for the specified block or expression.

Example:

```
int x = int.MaxValue;
```

```
unchecked
```

```
{  
    x++;  
}
```

```
Console.WriteLine(x);
```

Instead of throwing an OverflowException, the value can wrap around according to the rules of the integer type.

This is the opposite of using a checked context, where an overflow can result in an OverflowException. The notes connect unchecked with explicit casting and overflow handling.

7-**Parse and Null**

The **Parse** method is used to convert a string into a specific data type. It is especially common when receiving input from the user because Console.ReadLine() returns a string.

For example:

```
string ageInput = Console.ReadLine();
```

```
int age = int.Parse(ageInput);
```

Here, the input is initially a string, and `int.Parse()` converts that string into an integer.

Similarly, different data types provide their own `Parse` methods:

```
int age = int.Parse("20");
```

```
decimal salary = decimal.Parse("5000");
```

```
double value = double.Parse("10.5");
```

However, `Parse` expects the string to contain a valid value for the target data type. If the input is not in the correct format, an exception such as `FormatException` can occur.

The notes specifically identify the relationship between **Parse and null** as a topic for further study.

8-Convert and Null

The **Convert** class is another way to convert values from one data type to another. Unlike `Parse`, which is mainly used to convert strings into specific data types, `Convert` provides methods for converting between many different types.

For example:

```
string ageInput = Console.ReadLine();
```

```
int age = Convert.ToInt32(ageInput);
```

```
string salaryInput = Console.ReadLine();
```

```
decimal salary = Convert.ToDecimal(salaryInput);
```

The `Convert` class is part of the .NET Base Class Library (BCL), and it provides several methods such as:

```
Convert.ToInt32()
```

```
Convert.ToDouble()
```

```
Convert.ToDecimal()
```

```
Convert.ToString()
```

Convert.ToBoolean()

One important difference is that Convert has specific behavior when dealing with null, depending on the conversion method. The course notes specifically mark **Convert with null** as a topic for further study rather than explaining the complete behavior.

9-Convert vs Parse – Performance

Both **Parse** and **Convert** can be used for converting input values, especially strings to numeric data types, but they are not exactly the same.

Parse is directly associated with the target data type. For example:

```
int age = int.Parse("20");
```

While Convert uses the general-purpose Convert class:

```
int age = Convert.ToInt32("20");
```

The main differences between them involve their **behavior, supported conversions, handling of null, and performance**. Parse is generally a direct operation provided by the target type, while Convert is a more general conversion mechanism provided by the .NET BCL.

For normal applications, the performance difference is usually less important than choosing the method that matches the situation. However, the course notes specifically mention **comparing Convert and Parse in terms of performance** as a topic for further investigation.